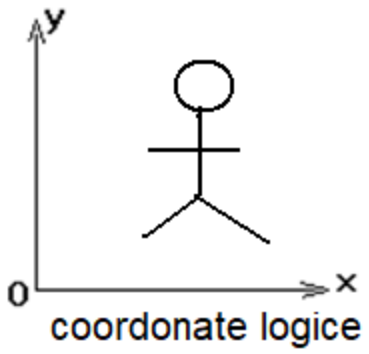
Transformarea de vizualizare 2D

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

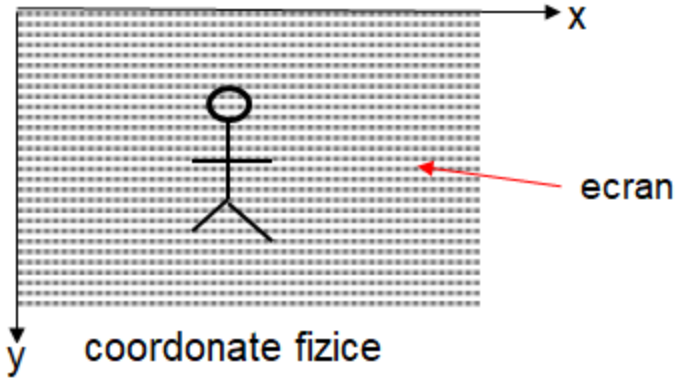
Coordonate logice şi coordonate fizice

- Desenele reprezentate într-un program de aplicatie grafica 2D sunt, de regula, raportate la un sistem de coordonate diferit de cel al suprafetei de afisare.
- Exemple: planul unei case, un desen tehnic, graficul unei functii, etc.



(metri, milimetri, viteza, timp, etc.)

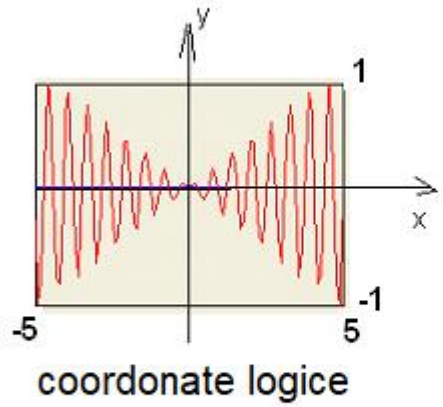
Sistemul de coordonate utilizator

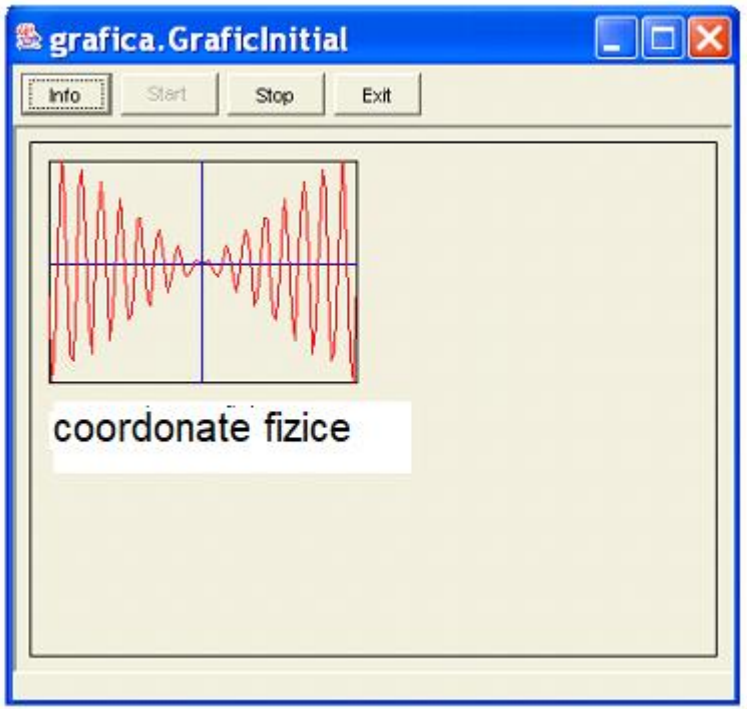


(pixeli, milimetri, s.a.)

Sistemul de coordonate dispozitiv

Transformarea coordonate logice - coordonate fizice



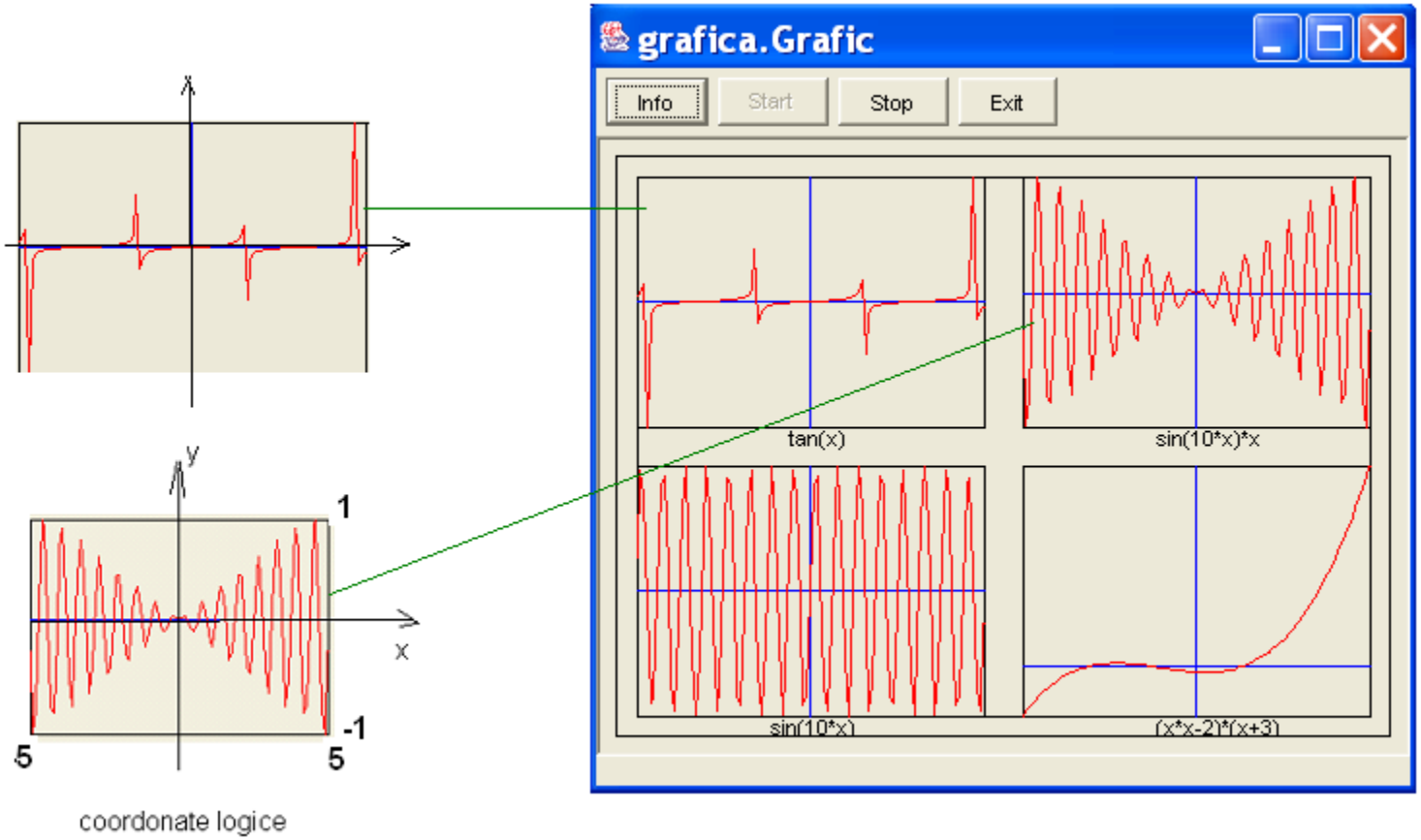


Aplicatia defineste graficul functiei in sistemul de coordonate utilizator (logice).

Coordonatele punctelor de pe graficul afisat pe ecran sunt adrese de pixeli.

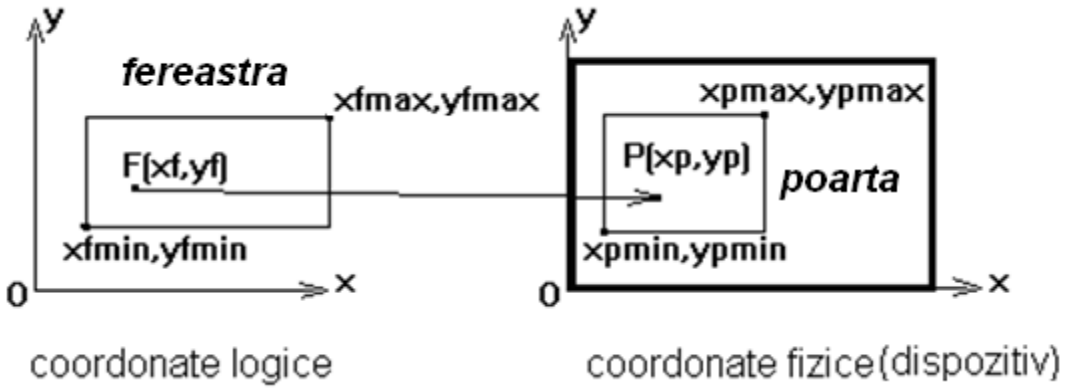
Coordonatele logice trebuie transformate in coordonate fizice.

Transformarea de vizualizare 2D (1)



Este necesara o transformare generala, prin care un desen definit in coordinate logice sa poata fi afisat intr-o zona oarecare a ecranului.

Transformarea de vizualizare 2D (2)
- transformarea "fereastra-poarta"-



- fereastra: zona din spatiul coord. logice al carei continut se va afisa
- poarta: zona din spatiul coord. fizice in care se va afisa continutul ferestrei

▪ Transformarea este definita folosind 2 dreptunghiuri, definite in cele 2 sisteme de coordonate, numite: ***fereastra (de vizualizare)*** si ***poarta (de afisare).***

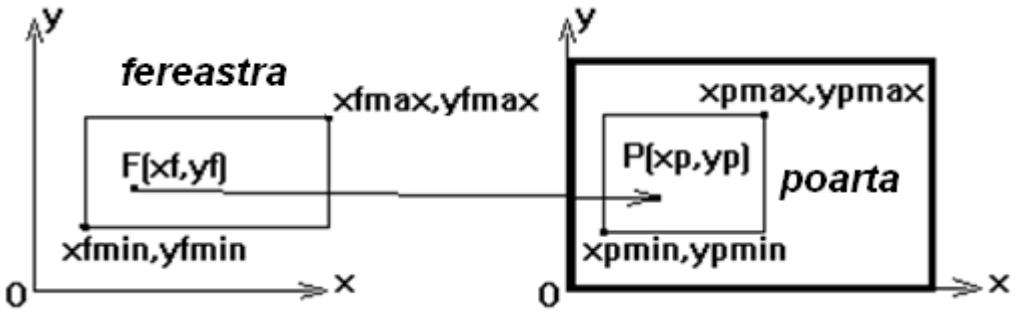
• Transformarea se mai numeste ***transformarea fereastra-poarta.***

F: un punct din fereastra.
P: punctul in care se transforma F prin transformarea fereastra-poarta.

➢Transformarea fereastra-poarta trebuie sa asigure ca:

" pozitia relativa a lui P in poarta de afisare este aceeasi cu pozitia relativa a lui F in fereastra".

Transformarea de vizualizare 2D (3)



$$\frac{xp - xp\,\min}{xp\,\max - xp\,\min} = \frac{xf - xf\,\min}{xf\,\max - xf\,\min}$$

$$\frac{yp - yp\,\min}{yp\,\max - yp\,\min} = \frac{yf - yf\,\min}{yf\,\max - yf\,\min}$$

$$sx = \frac{xp\,\max - xp\,\min}{xf\,\max - xf\,\min}$$

$$sy = \frac{yp\,\max - yp\,\min}{yf\,\max - yf\,\min}$$

- sx, sy reprezinta factorii de scalare ai transformarii
- tx, ty depind de pozitiile celor 2 ferestre fata de originea sistemului de coord. in care sunt definite

şi $tx = xpmin - sx*xfmin$ $ty = ypmin - sy*yfmin$

$$xp = xf * sx + tx$$
$$yp = yf * sy + ty$$

← Exprimarea matematica a transformarii fereastra – poarta, considerand o aceeasi orientare a axelor celor 2 sisteme de coordonate.

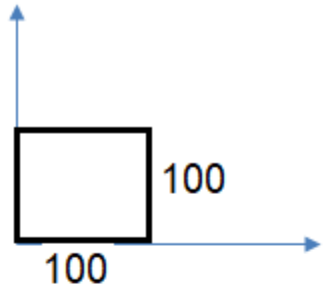
Transformarea de vizualizare 2D (4)

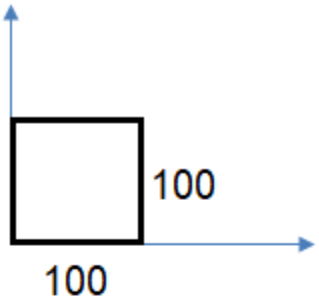
$$sx = \frac{xp\max - xp\min}{xf\max - xf\min}$$

$$sy = \frac{yp\max - yp\min}{yf\max - yf\min}$$

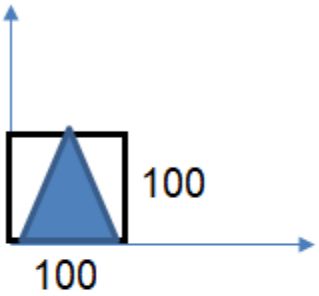
*tx = xpmin- sx*xfmin*

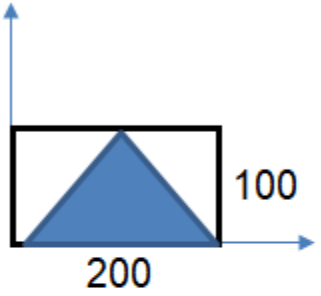
*ty= ypmin- sy*yfmin*



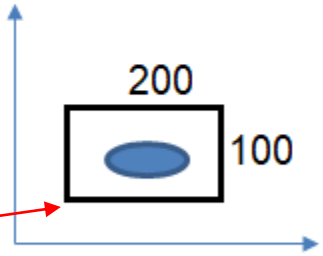


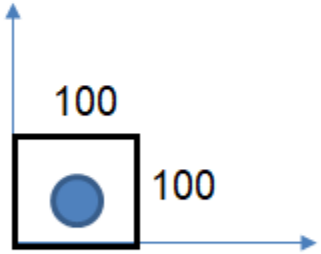
sx=1, sy = 1
tx = 0, ty =0



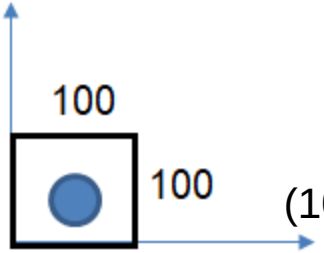


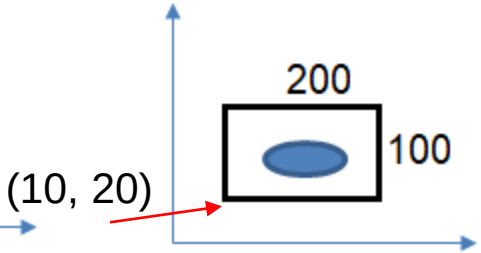
sx=2, sy = 1
tx = 0, ty =0





sx=0.5, sy = 1
tx = 0 - 0.5*10 = - 5
ty = 0 – 1*20 = -20

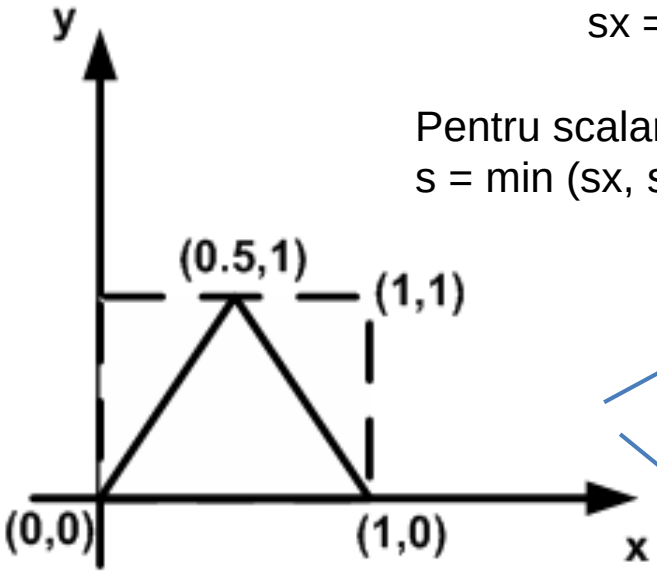




sx=2, sy = 1
tx = 10
ty = 20

Transformarea de vizualizare 2D (5)

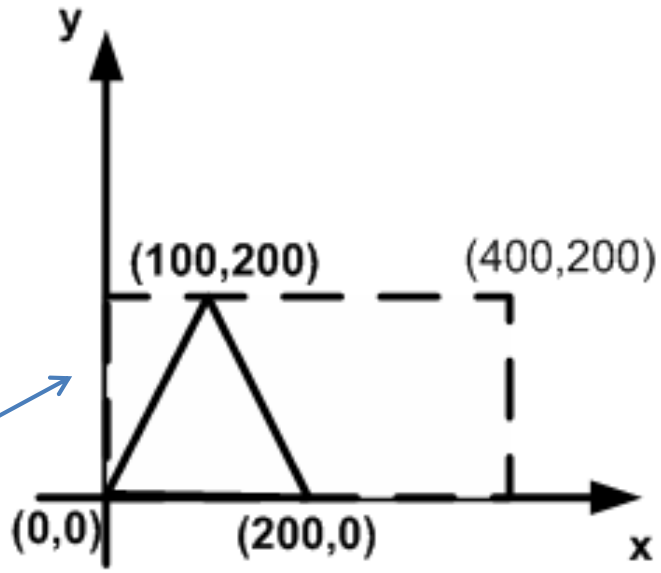
Scalare uniforma
cu afisare centrata in poarta de vizualizare



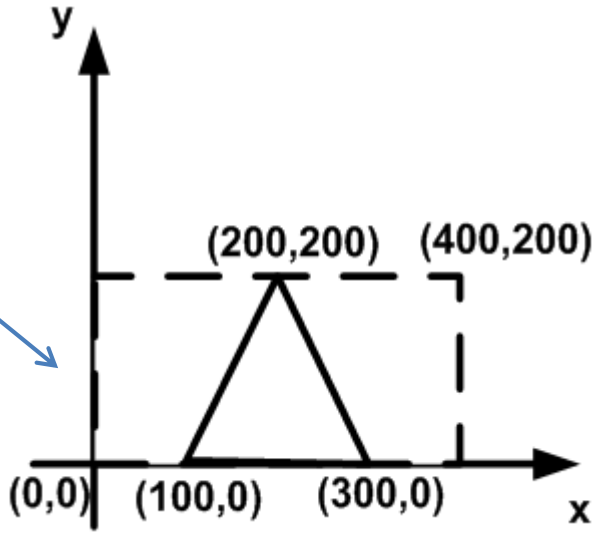
sx = 400, sy = 200

Pentru scalare uniforma:
s = min (sx, sy)

tx=0, ty =0



tsx = 100,
tsy =0



tsx =(xpmax – xpmin – s*(xfmax – xfmin))/2

= (400 – 200*1) /2 = 100

tsy =(ypmax – ypmin – s*(yfmax – yfmin))/2

= (200 – 200*1) /2 = 0

Transformarea de vizualizare 2D (7)

Efectele transformarii

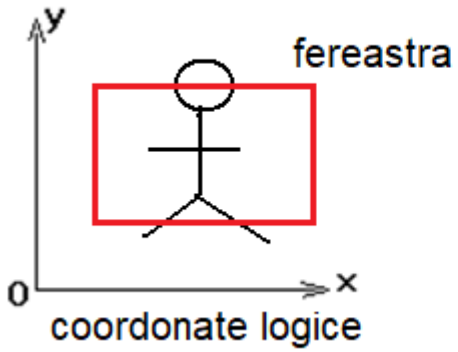
- Marire / micsorare, in functie de dimensiunile ferestrei si ale portii
- Deformare: fereastra si poarta nu sunt dreptunghiuri asemenea
- Pentru scalare uniforma: s = min(sx,sy)
- Afisare centrata in poarta: translatie suplimentara pe axa ox sau pe axa oy:

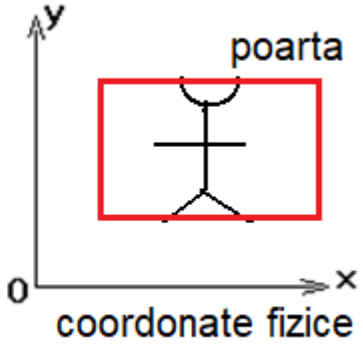
  tsx =(xpmax – xpmin – s*(xfmax – xfmin))/2

  tsy =(ypmax – ypmin – s*(yfmax – yfmin))/2

  Translatia totala: tx+tsx, ty+tsy

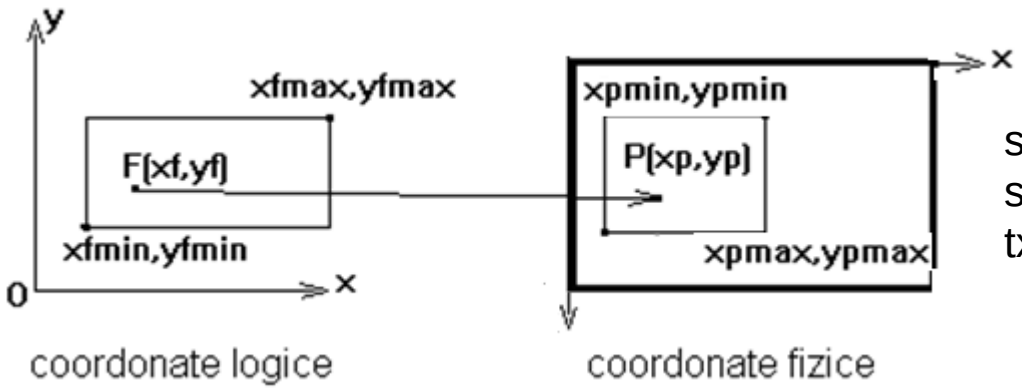
- Decuparea primitivelor aflate in afara ferestrei vizuale

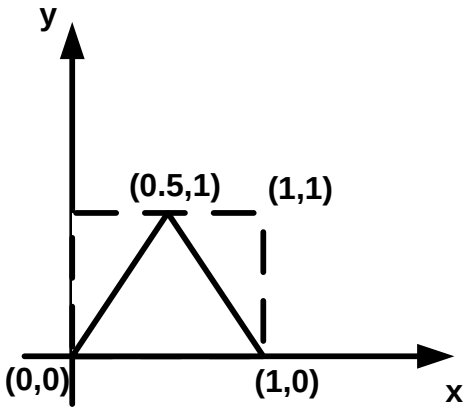




Transformarea de vizualizare 2D (8)

Corectia coordonatei yp
tinand cont de orientarea axelor sist. coord fizice





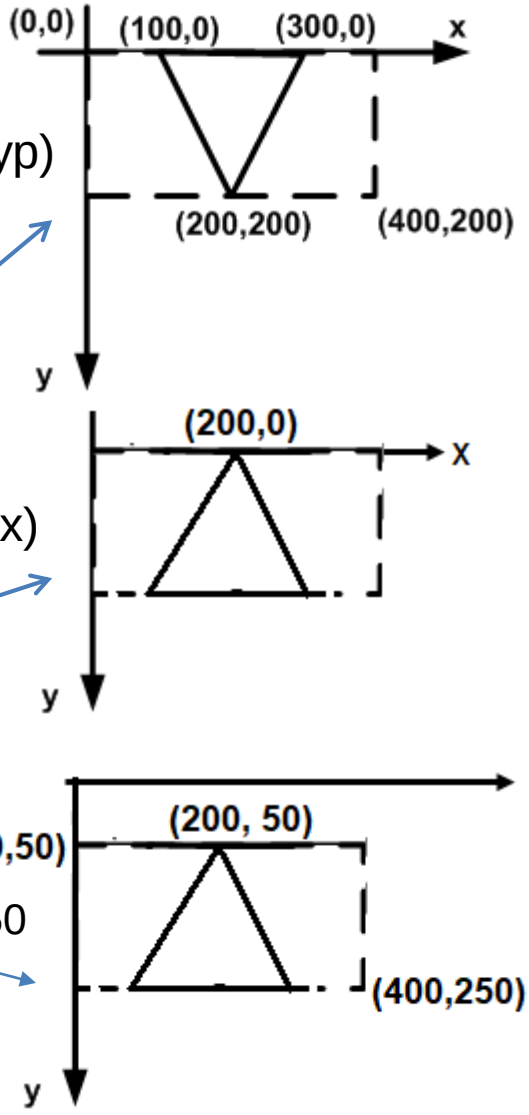
(xp, yp)

sx = 400, sy = 200
s=smin = 200
tx = tsx =100, ty =0

(xp, yp'= -yp+ypmax)

ty = 50

yp' = -yp + ypmax + ypmin



Transformarea de vizualizare 2D (9)

Formulele transformarii fereastra – poarta atunci cand axa OY a sistemului de coordinate fizice este orientata in jos:

xp = xf * sx + tx
yp = – (yf * sy + ty) + ypmin + ypmax = -yf * sy + ypmin + ypmax -ty

Exprimarea matriciala a transformarii fereastra-poarta in cazul corectiei coordonatei y

$$\begin{bmatrix} xp \\ yp \\ 1 \end{bmatrix} = M * \begin{bmatrix} xf \\ yf \\ 1 \end{bmatrix}$$

$$M = \begin{bmatrix} sx & 0 & tx \\ 0 & -sy & yp\min + yp\max - ty \\ 0 & 0 & 1 \end{bmatrix}$$

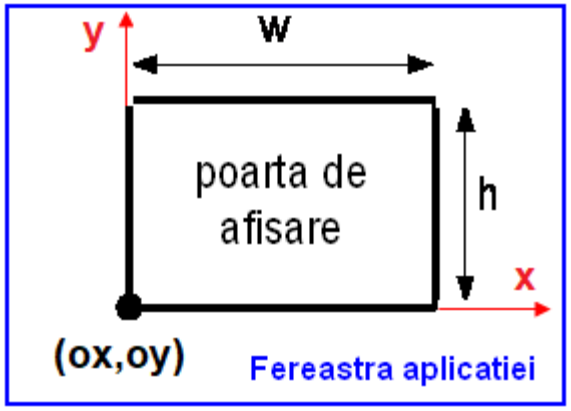
Implementarea transformarii de vizualizare în OpenGL (1)

- Pentru setarea zonei de afisare, în OpenGL se apeleaza

  glViewport(ox, oy, width, height);

unde (ox,oy) sunt coord. coltului stanga jos al viewportului, in coordonatele ferestrei aplicatiei.

- Orientarea axelor viewportului este aceeasi cu a sistemului coordonatelor logice:
  - Nu este necesara corectia coordonatei y → matricea transformarii de vizualizare este:



$$M = \begin{bmatrix} sx & 0 & tx \\ 0 & sy & ty \\ 0 & 0 & 1 \end{bmatrix}$$

- Se poate defini un desen in coordonate logice.
- Pentru obtinerea desenului in coordonate ecran se foloseste ca matrice de transformare a vârfurilor matricea transformării de vizualizare

Implementarea transformarii de vizualizare in OpenGL (2)

```
class TransformareaFereastraPoartaPatrate : public SimpleScene
{
……..
    protected:
    float length;
    ViewportSpace viewSpace; // dreptunghiul “poarta de afisare”
    LogicSpace logicSpace; // dreptunghiul “fereastra”
    glm::mat3 modelMatrix,
          visMatrix; //matricea transformarii de vizualizare
    bool uniform; // transformare uniforma/neuniforma
 // figura desenata se obtine prin rotatia unui patrat  in jurul centrului ferestrei
    float raza; // raza cercului pe care se roteste patratul
    float cx, cy; // centrul cercului
 };
```

Implementarea transformarii de vizualizare in OpenGL (3)

```
void TransformareaFereastraPoartaPatrate::Init()
{ ……..
   logicSpace.x = -5; // logic x
   logicSpace.y = -5; // logic y
   logicSpace.width = 10; // logic width
   logicSpace.height = 10; // logic height
   length = 2.0f; //latura patratului
   raza = 3.6f;

   cx = logicSpace.x + logicSpace.width / 2;
   cy = logicSpace.y +  logicSpace.height / 2;


   glm::vec3 corner = glm::vec3(cx - length / 2 + raza , cy - length / 2, 0);

   Mesh* square = Object2DPatrate::CreateSquare("square", corner, length, glm::vec3(1, 0, 0));

   AddMeshToList(square); // patratul care se roteste


   glm::vec3 cornerw = glm::vec3(logicSpace.x, logicSpace.y, 0);
   Mesh* window =
        Object2DPatrate::CreateSquare("window", cornerw, logicSpace.width, glm::vec3(0, 0, 0));
   AddMeshToList(window); // cadrul ferestrei

   uniform = true;
   }
```

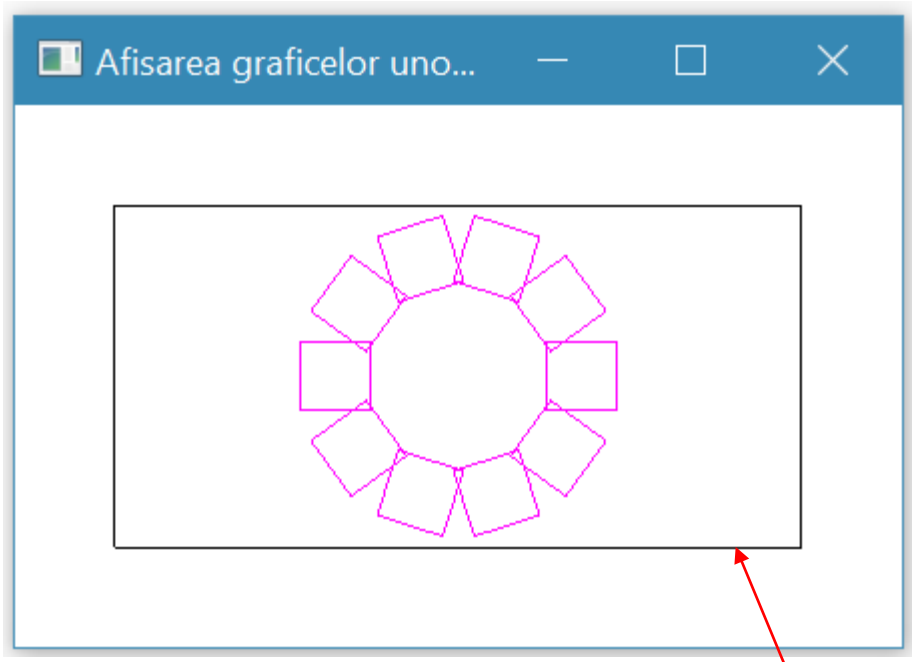
Implementarea transformarii de vizualizare in OpenGL (4)

Calculul matricei transformarii de vizualizare

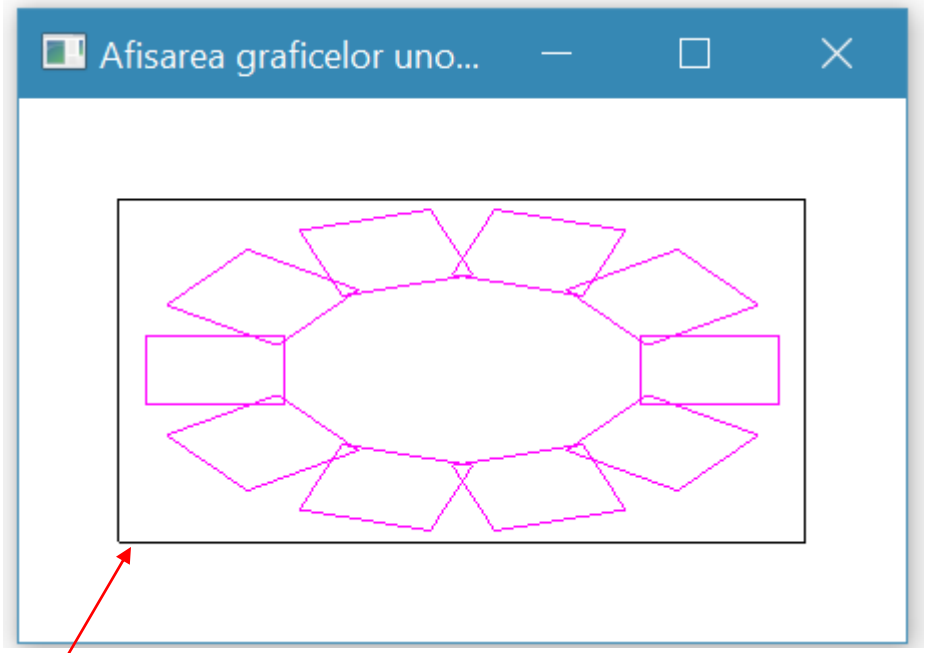
```
glm::mat3 TransformareaFereastraPoartaPatrate::VisualizationTransf2D
                (const LogicSpace & logicSpace, const ViewportSpace & viewSpace)
{   float sx, sy, tx, ty;
    sx = viewSpace.width / logicSpace.width;  sy = viewSpace.height / logicSpace.height;
    tx = viewSpace.x - sx * logicSpace.x;  ty = viewSpace.y - sy * logicSpace.y;
    return glm::transpose(glm::mat3(sx, 0.0f, tx,0.0f, sy, ty,0.0f, 0.0f, 1.0f));
}

glm::mat3 TransformareaFereastraPoartaPatrate::VisualizationTransf2DUnif(const LogicSpace
                                        & logicSpace, const ViewportSpace & viewSpace)
{   float sx, sy, tx, ty, smin;
    sx = viewSpace.width / logicSpace.width; sy = viewSpace.height / logicSpace.height;
    if (sx < sy) smin = sx;
        else smin = sy;
    tx = viewSpace.x - smin * logicSpace.x + (viewSpace.width - smin * logicSpace.width) / 2;
    ty = viewSpace.y - smin * logicSpace.y + (viewSpace.height - smin * logicSpace.height) / 2;
    return glm::transpose(glm::mat3(smin, 0.0f, tx,0.0f, smin, ty,.0f, 0.0f, 1.0f));
}
```

Transformare uniformă/neuniformă



Desenul cu transformare uniforma



Desenul cu transformare neuniforma

Cadrul ferestrei mapat pe poarta de afisare
(neafectat de transformarea uniforma)

Transformare uniformă/neuniformă - implementare

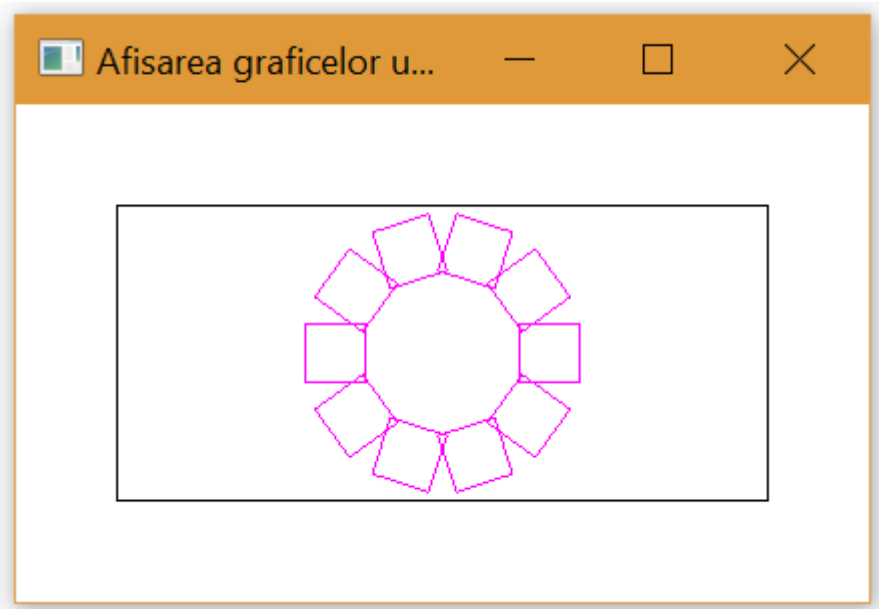
```
void TransformareaFereastraPoartaPatrate::Update(float deltaTimeSeconds)
{   glm::ivec2 resolution = window->GetResolution();
//setare poarta de vizualizare
    viewSpace = ViewportSpace(50, 50, resolution.x - 100, resolution.y - 100);
// seteaza zona ecran in care se va desena
    SetViewportArea(viewSpace, glm::vec3(1), true);
    visMatrix = VisualizationTransf2D(logicSpace, viewSpace);
//deseneaza cadrul ferestrei
    RenderMesh2D(meshes["window"], visMatrix, glm::vec3(0, 0, 0));
    if (uniform)   //modificata la apasarea tastei “s”
      visMatrix = VisualizationTransf2DUnif(logicSpace, viewSpace);
    else
      visMatrix = VisualizationTransf2D(logicSpace, viewSpace);
//deseneaza patratele rotite
   DrawRotatedSquares(visMatrix, glm::vec3(1, 0, 1), 10);
}
```

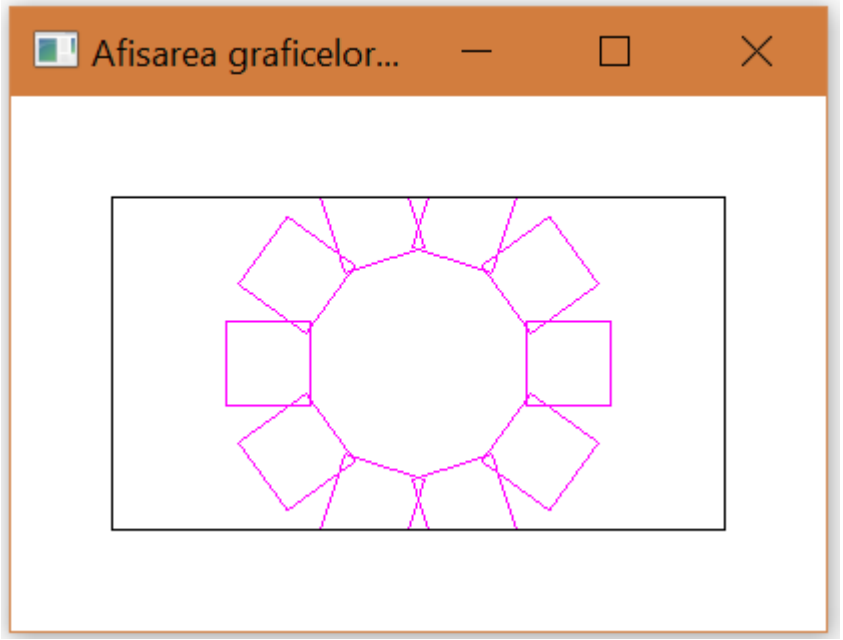
Transformare uniformă/neuniformă - implementare

```
void TransformareaFereastraPoartaPatrate::DrawRotatedSquares(glm::mat3
            visMatrix, glm::vec3 color, int iterations)
{
    float delta = 2 * 3.14159 / iterations;

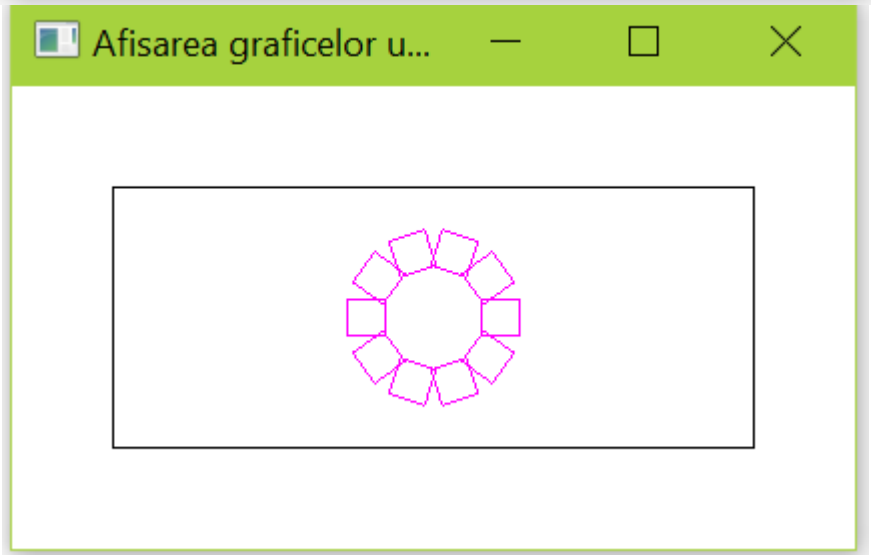
    for (int i = 0; i < iterations; i++)
   {
       modelMatrix = visMatrix *
                          Transform2D::Translate(cx,cy ) *
                          Transform2D::Rotate(delta * i) *
                          Transform2D::Translate(-cx,-cy);
       RenderMesh2D(meshes["square"], modelMatrix, color);
   }
}
```

Mărirea/Micşorarea ferestrei





Fereastra micşorată.
Efectul de mărire a desenului şi decupare la marginile ferestrei



Fereastra mărită.
Efectul de micsorare a desenului

Mărirea/Micşorarea ferestrei - implementare

```
void TransformareaFereastraPoartaPatrate::Update(float deltaTimeSeconds)
{
   glm::ivec2 resolution = window->GetResolution();
  // logicSpace.x = logicSpace.x - 2; logicSpace.y = logicSpace.y -2;
  // logicSpace.width = logicSpace.width + 4; logicSpace.height = logicSpace.height +4;
  //Micsorare
   logicSpace.x = logicSpace.x + 1; logicSpace.y = logicSpace.y +1;
   logicSpace.width = logicSpace.width – 2; logicSpace.height = logicSpace.height -2;

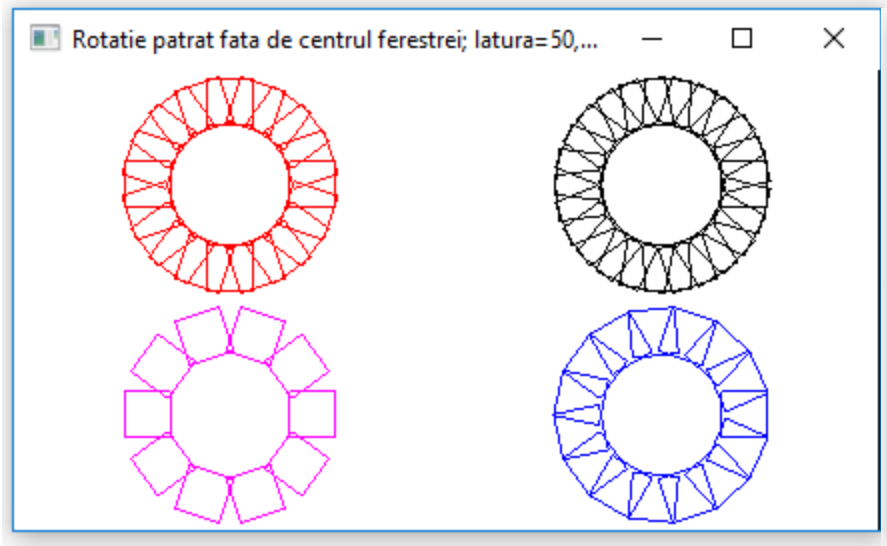
   glm::vec3 cornerw1 = glm::vec3(logicSpace.x, logicSpace.y, 0);
   Mesh* window1 = Object2DPatrate::CreateSquare("window1", cornerw1, logicSpace.width,
                           glm::vec3(0, 0, 0));
    AddMeshToList(window1);

   viewSpace = ViewportSpace(50, 50, resolution.x - 100, resolution.y - 100);
   SetViewportArea(viewSpace, glm::vec3(1), true);

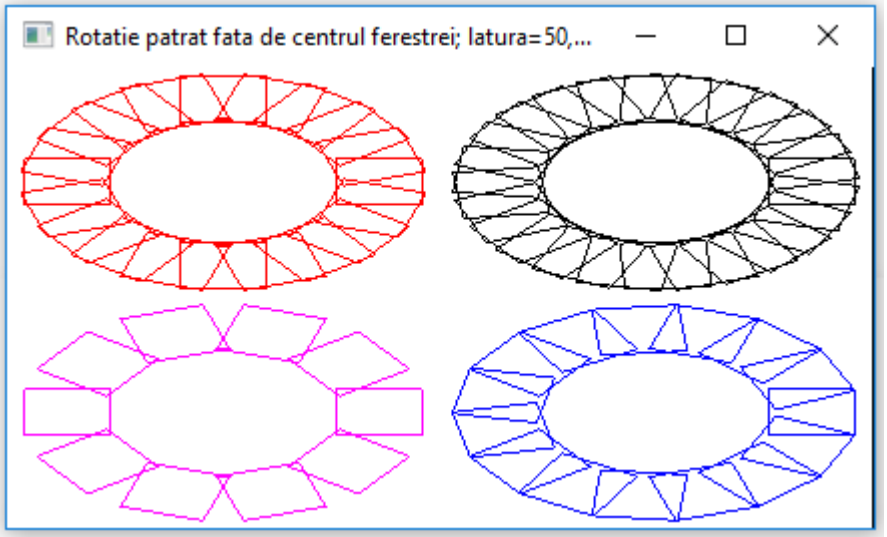
   visMatrix = VisualizationTransf2D(logicSpace, viewSpace);
   RenderMesh2D(meshes["window1"], visMatrix, glm::vec3(0, 0, 0));
   if (uniform)
              visMatrix = VisualizationTransf2DUnif(logicSpace, viewSpace);
   else     visMatrix = VisualizationTransf2D(logicSpace, viewSpace);

   DrawRotatedSquares(visMatrix, glm::vec3(1, 0, 1), 10);
}
```

Desenare în 4 porţi de afişare



Transformare uniforma



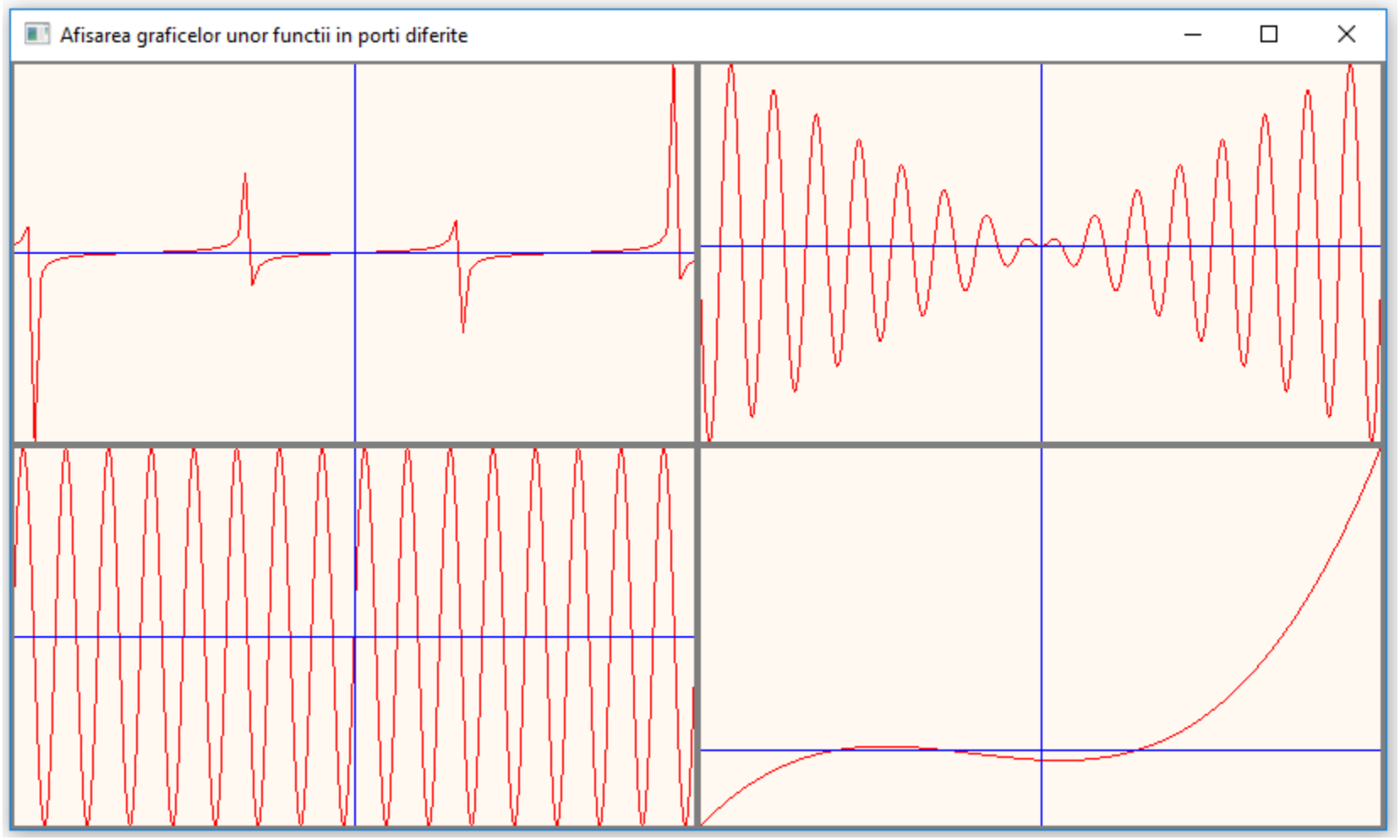
Transformare neuniforma

Desenare în 4 porţi de afişare - implementare

```
void TransformareaFereastraPoartaPatrate::Update(float deltaTimeSeconds)
{
   glm::ivec2 resolution = window->GetResolution(); //fereastra aplicatiei

   // desenare in partea stanga-jos a ferestrei aplicatiei
   viewSpace = ViewportSpace(0, 0, resolution.x / 2, resolution.y / 2);
   SetViewportArea(viewSpace, glm::vec3(1), true);
    if (uniform) visMatrix = VisualizationTransf2DUnif(logicSpace, viewSpace);
     else  visMatrix = VisualizationTransf2D(logicSpace, viewSpace);
   DrawRotatedSquares(visMatrix, glm::vec3(1, 0, 1), 10);
    // desenare in partea dreapta-jos a ferestrei aplicatiei
   viewSpace = ViewportSpace(resolution.x / 2, 0, resolution.x / 2, resolution.y / 2);
   SetViewportArea(viewSpace, glm::vec3(1), true);
   if (uniform) visMatrix = VisualizationTransf2DUnif(logicSpace, viewSpace);
   else visMatrix = VisualizationTransf2D(logicSpace, viewSpace);
   DrawRotatedSquares(visMatrix, glm::vec3(0, 0, 1), 15);
// analog pentru desenare in partea stanga-sus si dreapta-sus a ferestrei
}
```

Afişarea graficelor unor funcţii în porţi diferite



Afişarea graficelor unor funcţii în porţi diferite – implementare (1)

```
Mesh* Object2DGrafic::CreateFunctionPlot(std::string name,
        float &x1, float &x2, float &y1, float &y2, float pas, float(*f)(double f))
{   // calculeaza punctele de pe graficul functiei matematice y = f(x) in intervalul x1 – x2,
    //in coordonate logice
   // întoarce în (x1,y1), (x2,y2) încadrarea graficului incluzand si axele sistemului de coordonate
    float x, y, xmin=x1, xmax=x2;

   // calculeaza incadrarea graficului pe axa OY( [y1-y2]) in intervalul xmin - xmax
      y1 = y2 = f(xmin);
      for (x = xmin + pas; x<xmax; x += pas)
            {
                  if ((y = f(x))<y1) y1 = y;
                  if (y>y2)y2 = y;
            }

      if ((y = f(xmax))<y1) y1 = y;
      if (y>y2) y2 = y;
```

Afisarea graficelor unor functii in porti diferite – implementare (2)

```
// Include axele OX, OY în încadrarea graficului
   if (0<xmin)x1 = 0; else x1 = xmin;
   if (0<y1)y1 = 0;
   if (0>xmax)x2 = 0; else x2 = xmax;
   if (0>y2)y2 = 0;

Mesh* plot = new Mesh(name);
plot->SetDrawMode(GL_LINES); // se vor uni câte 2 varfuri succesive
std::vector<VertexFormat> vertices;
// memoreaza varfurile axei OX – se asociaza varfurilor culoarea albastru
    vertices.push_back(VertexFormat(glm::vec3(x1, 0, 0), glm::vec3(0, 0, 1)));
    vertices.push_back(VertexFormat(glm::vec3(x2, 0, 0), glm::vec3(0, 0, 1)));
// memoreaza varfurile axei OY – se asociaza varfurilor culoarea albastru
   vertices.push_back(VertexFormat(glm::vec3(0, y1, 0), glm::vec3(0, 0, 1)));
   vertices.push_back(VertexFormat(glm::vec3(0, y2, 0), glm::vec3(0, 0, 1)));
   std::vector<unsigned short> indices;
   indices.push_back(0); indices.push_back(1); indices.push_back(2);indices.push_back(3);
```

Afisarea graficelor unor functii in porti diferite – implementare (3)

```
// se adauga varfurile care aproximeaza graficul
// se asociaza varfurilor culoarea roşu
   int contor = 4;
   for (x = xmin; x<xmax - pas; x += pas)
    { vertices.push_back(VertexFormat(glm::vec3(x, f(x), 0), glm::vec3(1, 0, 0)));
      indices.push_back(contor); indices.push_back(contor+1);
      contor++;
     }
// se adauga ultimul varf
    vertices.push_back(VertexFormat(glm::vec3(xmax, f(xmax), 0), glm::vec3(1, 0, 0)));

    plot->InitFromData(vertices, indices);
   return plot;

//CreateFunctionPlot
}
```

Afisarea graficelor unor functii in porti diferite – implementare (4)

```
float f1(double x)
{
return (float)(sin(10 * x)*x);
}

float f2(double x)
{
return (float)(tan(x));
}

float f3(double x)
{
return (float)(sin(10 * x));
}

float f4(double x)
{
return (float)((x * x - 2) * (x + 3));
}
```

Afisarea graficelor unor functii in porti diferite – implementare (5)

```
void TransformareaFereastraPoartaGrafic::Init()
{
//setare camera
……….

// se deseneaza graficele celor 4 functii in intervalul xmin=-5, xmax=5
        float y1, y2, x1=-5.0f, x2 = 5.0f;
// se creaza un mesh cu graficul functiei f1; se seteaza spatiul logic pentru graficul functiei f1
        Mesh* mesh_f1 = Object2DGrafic::CreateFunctionPlot("mesh_f1", x1, x2, y1, y2, 0.01f, f1);
        AddMeshToList(mesh_f1);
        logicSpace1.x = x1;
        logicSpace1.width = x2 - x1;
        logicSpace1.y = y1;
        logicSpace1.height = y2 - y1;
//se creaza, in mod analog, mesh_f2, mesh_f3, mesh_f4 şi logicSpace2, logicSpace3, logicSpace4
……..
}
```

Afisarea graficelor unor functii in porti diferite – implementare (6)

```
void TransformareaFereastraPoartaGrafic::Update(float deltaTimeSeconds)
{
    glm::ivec2 resolution = window->GetResolution(); // fereastra aplicatiei

// seteaza zona de desenare – partea stanga-sus
   viewSpace = ViewportSpace(0, resolution.y / 2, resolution.x / 2, resolution.y / 2);
   SetViewportArea(viewSpace, glm::vec3(1, 0.98, 0.95), true);
   visMatrix = VisualizationTransf2D(logicSpace2, viewSpace);
   DrawFunctionPlot(visMatrix, 2);//deseneaza gaficul functiei f2
// partea dreapta-sus a ferestrei aplicatiei
   viewSpace = ViewportSpace(resolution.x / 2, resolution.y / 2, resolution.x / 2, resolution.y / 2);
   SetViewportArea(viewSpace, glm::vec3(1,0.98,0.95), true); //culoarea fondului pt viewport: bej
   visMatrix = VisualizationTransf2D(logicSpace1, viewSpace);
   DrawFunctionPlot(visMatrix, 1); //deseneaza graficul functiei f1
// analog pentru desenarea celorlalte functii
}
```

Afisarea graficelor unor functii in porti diferite – implementare (7)

```
void TransformareaFereastraPoartaGrafic::DrawFunctionPlot(glm::mat3 visMatrix, int nr_functie)
{
    modelMatrix = visMatrix;
    switch (nr_functie)
    {
     case 1:
        RenderMesh2D(meshes["mesh_f1"],shaders["ShaderGrafic"],modelMatrix);
        break;
     case 2:
        RenderMesh2D(meshes["mesh_f2"], shaders["ShaderGrafic"], modelMatrix);
        break;
     case 3:
        RenderMesh2D(meshes["mesh_f3"], shaders["ShaderGrafic"], modelMatrix);
        break;
     case 4:
        RenderMesh2D(meshes["mesh_f4"], shaders["ShaderGrafic"], modelMatrix);
        break;
     default:
        break;
    }
}
```